

File IO and MVC

Bilingual reference tables

Table 1: Core concepts

Chinese (中文)	English	Description
文件流	File stream	<code>ifstream</code> / <code>ofstream</code> attach your program to a disk file.
文本模式	Text mode	Formatted I/O; newline translation may happen on some OS.
二进制模式	Binary mode	Raw bytes via <code>read</code> / <code>write</code> ; use for exact layouts.
序列化	Serialization	Write logical state to a file; load reconstructs it.
版本标记	Version stamp	First number in a save file for format evolution.
MVC 模型	Model (MVC)	Data + rules; no drawing.
MVC 视图	View (MVC)	Renders state; read-only access to model.
MVC 控制器	Controller (MVC)	Maps input to model operations.

Table 2: Symbols & APIs

Symbol / API	English	Chinese (中文)	Usage
<code>std::ifstream</code>	Input file stream	输入文件流	Read file; test <code>if (!in)</code> .
<code>std::ofstream</code>	Output file stream	输出文件流	Write file; default truncates.
<code><<</code> / <code>>></code>	Insert / extract	插入 / 提取	Text formatted I/O.
<code>read</code> / <code>write</code>	Unformatted I/O	无格式读写	Binary buffers; <code>char*</code> cast.
<code>std::getline</code>	Read a line	读一行	Use for strings that may contain spaces.
<code>ignore()</code>	Skip characters	跳过输入	After <code>>></code> , before <code>getline</code> .

Table 3: Chapter terms

English	Chinese (中文)	Explanation
Working directory	工作目录	Relative paths resolve from where you launch the program.
Trivially copyable	可平凡拷贝	Safe for <code>write</code> / <code>read</code> as bytes; not <code>std::string</code> .
FEN	局面记号法	One-line text description of a chess position (example lab).

Part A — Why files?

Problem: RAM disappears when the process exits — a puzzle best time or Elo rating must live on disk.

Solution: File streams write **persistent** bytes.

Part A1 — File I/O basics: concepts and syntax (15 min)

What is a file stream?

A **file stream** is a C++ object that acts like a pipe between your program (in RAM) and a file (on disk). You can:

- **Write** data from program variables → disk file (save)
- **Read** data from disk file → program variables (load)

Think of it like a water pipe: water (data) flows from one side to the other.

C++ file I/O syntax quick reference

(Expanded detail beyond the opening bilingual tables; the rest of this document is English-only.)

Core file stream classes

Concept	Syntax	Description

Output file stream (write)	<code>std::ofstream</code>	Writes data to a file.
Input file stream (read)	<code>std::ifstream</code>	Reads data from a file.
Read/write file stream	<code>std::fstream</code>	Supports both reading and writing on the same file.

File open modes (`std::ios::...`)

Concept	Flag	Role
Binary mode	<code>std::ios::binary</code>	Raw bytes; no newline translation or text escaping.
Open at end	<code>std::ios::ate</code>	Seek to the end of the file immediately after open.
Append	<code>std::ios::app</code>	Every write appends at the end of the file.
Truncate	<code>std::ios::trunc</code>	Discard existing file content when opening for output.
Input	<code>std::ios::in</code>	Open for reading.
Output	<code>std::ios::out</code>	Open for writing.

| Note: combine modes with bitwise OR, for example `ios::ate | ios::binary` .

Common member functions

Concept	Syntax	Role
Binary block write	<code>stream.write(ptr, len)</code>	Write <code>len</code> bytes from memory to the file.
Binary block read	<code>stream.read(ptr, len)</code>	

		Read <code>len</code> bytes from the file into memory.
Read position	<code>stream.tellg()</code>	Current read position (e.g. file size if opened at end).
Write position	<code>stream.tellp()</code>	Current write position.
Seek read pointer	<code>stream.seekg(offset, origin)</code>	Move the read pointer.
Seek write pointer	<code>stream.seekp(offset, origin)</code>	Move the write pointer.
Open succeeded?	<code>stream.is_open()</code>	Whether the file was opened successfully.
At end of file?	<code>stream.eof()</code>	Whether the read pointer is at EOF.
Close	<code>stream.close()</code>	Release the file (often optional when the stream is destroyed).

Seek origins

Concept	Flag	Meaning
Beginning	<code>std::ios::beg</code>	Offset from the start of the file.
Current	<code>std::ios::cur</code>	Offset from the current position.
End	<code>std::ios::end</code>	Offset from the end of the file.

Casts and `sizeof`

Concept	Syntax	Role
Reinterpret pointer to bytes	<code>reinterpret_cast<char*></code> (or <code>const char*</code>)	View memory as raw bytes for <code>read</code> / <code>write</code> .
Size in bytes	<code>sizeof(expr)</code> or <code>sizeof(T)</code>	How many bytes to read or write for a POD value.

Text I/O operators

Concept	Syntax	Role
Insert (write)	<code>stream << value</code>	Formatted text output.
Extract (read)	<code>stream >> variable</code>	Formatted text input.

Typical code templates

Binary file write

Code block

```
1  std::ofstream f("a.bin", std::ios::binary);
2  f.write(reinterpret_cast<const char*>(&val), sizeof(val));
```

Get Binary File Size

Code block

```
1  std::ifstream f("a.bin", std::ios::ate | std::ios::binary);
2  long long fileSize = f.tellg();
```

The basic workflow (4 steps)

Code block

```
1  #include <fstream>    // Step 0: include header
2  #include <iostream>
3
4  int main() {
5      // Step 1: Create stream object (opens file automatically)
6      std::ofstream out("data.txt");    // Output stream, creates/truncates
                                         "data.txt"
7
8      // Step 2: Check if open succeeded (CRITICAL!)
9      if (!out) {                      // If open failed
10         std::cerr << "Failed to open file!\n";
11         return 1;                     // Exit with error
12     }
13
14     // Step 3: Write data (like using std::cout)
15     out << "Hello, file!" << std::endl; // Writes to file, not screen
16     out << 42 << '\n';                 // Numbers become text
17
18     // Step 4: Close (automatic via destructor, but can be explicit)
19     out.close();                     // Optional: file closes when 'out'
                                         goes out of scope
20
21     return 0;
22 }
```

Reading from a file

Code block

```
1  #include <fstream>
2  #include <iostream>
3  #include <string>
4
5  int main() {
6      // Step 1: Create input stream
7      std::ifstream in("data.txt");    // Tries to open existing file
8  }
```

```

9      // Step 2: Check if file exists and is readable
10     if (!in) {                                // Also: if (!in.is_open())
11         std::cerr << "Cannot open data.txt (file may not exist)\n";
12         return 1;
13     }
14
15     // Step 3: Read data (like using std::cin)
16     std::string word;
17     int number;
18
19     in >> word;                                // Reads one word (stops at whitespace)
20     in >> number;                              // Reads a number, converts from text to int
21
22     std::cout << "Read: " << word << " and " << number << '\n';
23
24     // Step 4: Check for errors during reading
25     if (in.fail()) {
26         std::cerr << "Error: expected number but got something else\n";
27     }
28
29     return 0;
30 } // File closes automatically

```

Key concepts explained

Concept	Explanation	Code example
---------	-------------	--------------

| **Constructor opens file** | When you create `ifstream` / `ofstream` with a filename, it tries to open immediately | `std::ifstream in("file.txt");` |

| **is_open()** | Returns `true` if file successfully opened | `if (in.is_open()) { /* safe to read */ }` |

| ****Operator `!`**** | Tests if stream is in bad state (failed to open or error) | `if (!in) { /* handle error */ }` |

| **Destructor closes file** | When stream object goes out of scope, file auto-closes (RAII) | No manual close needed |

| **fail()** | Returns `true` if last operation failed (wrong type, EOF, etc.) | `if (in.fail()) { /* error */ }` |

| **eof()** | Returns `true` if end-of-file reached | `while (!in.eof()) { /* read */ }` |

Text mode vs Binary mode (concept)

Text mode (default):

- Data is formatted as human-readable text
- `42` becomes characters `'4'` and `'2'` (2 bytes)
- Newline characters `\n` may be converted to `\r\n` on Windows
- Use `<<` and `>>` operators

Binary mode:

- Data written exactly as bytes in memory
- `42` as `int` is 4 bytes (binary representation)
- No newline conversion — exact bytes preserved
- Use `read()` and `write()` member functions

Common beginner mistakes

Mistake	Why it fails	Correct approach
Forget to check if file opened	Silent failure, garbage data	Always <code>if (!stream)</code> after opening
Reading with <code>>></code> for full names	Stops at first space	Use <code>std::getline(stream, string)</code> for lines
Not including <code><fstream></code>	Compiler error	Always <code>#include <fstream></code>
Using file after error	Undefined behavior	Check <code>fail()</code> or test stream in while loop

Example 1 — Minimal: one integer (`ex01\_one\_number.cpp`)

Chess link: store the player's last **rating** as one number.

Build: `g++ -std=c++17 -Wall -o ex01 ex01\_one\_number.cpp`

Code block

```
1  // ex01_one_number.cpp – Minimal: write one integer, read it back.
2  #include <fstream>    // KEY: Must include this header to use file streams
3  #include <iostream>
4
5  int main() {
6      const char* path = "score_ex01.txt"; // filename as const char*
7
8      {
9          // [KEY SYNTAX] ofstream constructor auto-opens file (write mode,
          // creates if missing, truncates if exists)
10         std::ofstream out(path);
11
12         // [KEY CHECK] if (!out) is equivalent to if (out.fail()), checks if
          // open failed
13         if (!out) {
14             std::cerr << "Cannot write " << path << '\n';
15             return 1; // Must handle open failure, otherwise subsequent
          // writes are meaningless
16         }
17
18         // [KEY SYNTAX] << operator formats data as text, 42 becomes
          // characters '4' and '2'
19         out << 42;
20     } // [KEY] Right brace destroys 'out', automatically calls close()
21
22     // [KEY SYNTAX] ifstream for reading, constructor attempts to open file
23     std::ifstream in(path);
24     int n = 0;
25
26     // [KEY] in >> n returns reference to in, testable in if; false if EOF or
          // type error
27     if (in >> n)
28         std::cout << "Read: " << n << '\n';
29     else
30         std::cout << "Open or read failed\n";
31
32     return 0;
33 }
```

Key Syntax Summary:

Syntax	Meaning	Note
<code>#include <fstream></code>	Include file stream class definitions	Required, otherwise ifstream/ofstream unknown
<code>std::ofstream out(path)</code>	Create output stream and open file	Default ios::out, truncates existing file
<code>if (!out)</code>	Check open/write failure	Can also use <code>!out.is_open()</code>
<code>out << 42</code>	Formatted write	Becomes text "42", not 4-byte binary int
Auto-destructor close	RAII mechanism	Auto-closes when leaving scope, no manual close() needed
<code>in >> n</code>	Formatted read	Converts from text to int; fails if file content isn't numeric

Example 2 — Basic: many scores, one per line (`ex02\_highscore\_lines.cpp`)

Chess link: top puzzle completion times or tournament points.

Build: ``g++ -std=c++17 -Wall -o ex02 ex02_highscore_lines.cpp``

Code block

```
1 // ex02_highscore_lines.cpp
2 #include <fstream>
```

```

3  #include <iostream>
4  #include <vector>
5
6  void save_scores(const char* path, const std::vector<int>& scores) {
7      std::ofstream out(path);
8      if (!out) {                                // [KEY] Check inside function, don't assume
caller checked
9          std::cerr << "save failed\n";
10         return;                                // Early return, avoid subsequent no-op
11     }
12     // [KEY SYNTAX] Range-based for loop (C++11), works perfectly with vector
13     for (int s : scores)
14         out << s << '\n';                    // Write newline after each number, "one per
line"
15 }
16
17 std::vector<int> load_scores(const char* path) {
18     std::vector<int> v;
19     std::ifstream in(path);
20     if (!in)
21         return v;                                // [KEY] Return empty vector if file doesn't
exist, not an error
22
23     int x = 0;
24     // [KEY SYNTAX] while (in >> x) is idiomatic: continue if read succeeds,
stop on EOF/failure
25     while (in >> x)
26         v.push_back(x);                        // Dynamic append, vector auto-expands
27     return v;
28 }
29
30 int main() {
31     const char* path = "highscores_ex02.txt";
32     // [KEY SYNTAX] C++11 braced initializer list, passed directly to const
vector<int>& parameter
33     save_scores(path, {2500, 1800, 3200});
34
35     std::cout << "Loaded:";
36     for (int s : load_scores(path))
37         std::cout << ' ' << s;
38     std::cout << '\n';
39     return 0;
40 }

```

Key syntax summary:

Syntax	Meaning	Note
<code>std::vector<int>&</code>	Pass by reference to avoid copying	<code>const</code> means the function will not modify the vector.
<code>while (in >> x)</code>	Stream used as a condition	True while reads succeed; false at EOF or on failure.
<code>v.push_back(x)</code>	Append at the end	<code>vector</code> manages memory; no manual pre-allocation.
<code>{2500,1800,3200}</code>	Braced initializer list (C++11)	Temporary vector passed to a const reference parameter.
<code>out << s << '\n'</code>	Chained insertions	Left-associative, same as <code>(out << s) << '\n'</code> .
Early <code>return</code>	Common error-handling style	Avoids deep nesting of <code>if</code> .

Example 3 — Text vs binary size (`ex03\_text\_vs\_binary\_size.cpp`)

Idea: Same value `1000` — text characters vs four raw bytes.

Build: `g++ -std=c++17 -Wall -o ex03 ex03\_text\_vs\_binary\_size.cpp`

Code block

```

1  // ex03_text_vs_binary_size.cpp
2  #include <fstream>
3  #include <iostream>
4
5  int main() {
6      const int value = 1000;
7  
```

```

8      {
9          std::ofstream t("thousand_text.txt");
10         t << value; // [KEY] Text mode: 1000 becomes 4 characters
            '1', '0', '0', '0'
11     }
12     {
13         // [KEY SYNTAX] ios::binary flag, prohibits any character conversion
            (e.g., \n → \r\n)
14         std::ofstream b("thousand.bin", std::ios::binary);
15
16         // [KEY SYNTAX] reinterpret_cast forces int* to const char*, making
            write accept it
17         // Param 1: Memory start address (byte pointer)
18         // Param 2: Byte count (sizeof(int) is typically 4)
19         b.write(reinterpret_cast<const char*>(&value), sizeof(value));
20     }
21
22     // [KEY SYNTAX] ios::ate = at end, seek to end immediately after opening
23     // Combined with tellg() to get file size in bytes right away
24     std::ifstream t("thousand_text.txt", std::ios::ate | std::ios::binary);
25     std::ifstream bin("thousand.bin", std::ios::ate | std::ios::binary);
26
27     // [KEY SYNTAX] tellg() returns current read position (equals file size
            when opened with ate)
28     std::cout << "Text file bytes: " << t.tellg() << '\n';
29     std::cout << "Binary file bytes: " << bin.tellg() << '\n';
30     return 0;
31 }

```

Key syntax summary:

Syntax	Meaning	Note
<code>std::ios::binary</code>	Binary mode flag	Pass with the filename in the stream constructor.
<code>ios::ate</code>	Seek to end right after open	Often paired with <code>tellg()</code> to get file size in bytes.
<code>`</code>	<code>`</code> (bitwise OR)	Combine open flags
<code>reinterpret_cast</code>	Reinterpret pointer type	Low-level but needed here; only for trivial (POD)

		layouts.
<code>sizeof(value)</code>	Byte size at compile time	Often 4 for <code>int</code> , but platform-dependent.
<code>tellg()</code>	Returns <code>streampos</code>	“Tell get” : current read position.
<code>write(ptr, len)</code>	Raw byte write	No formatting; copies memory as bytes.

Comparison:

- Text file: `1000` is four bytes (characters `'1'`, `'0'`, `'0'`, `'0'`).
- Binary file: the `int` value 1000 is four bytes (typical little-endian: `0xE8 0x03 0x00 0x00`).
- Note: for tiny values text size can be smaller by coincidence; for large magnitudes binary is usually more compact.

Example 4 — Trap: `>>` then `getline` (`ex04\_extract\_then\_getline\_bug.cpp`)

Build: `g++ -std=c++17 -Wall -o ex04 ex04_extract_then_getline_bug.cpp``

Code block

```

1  // ex04_extract_then_getline_bug.cpp
2  #include <fstream>
3  #include <iostream>
4  #include <string>
5
6  int main() {
7      const char* path = "mixed_ex04.txt";
8      {
9          std::ofstream out(path);
10         out << "42\n";           // Write: number + newline
11         out << "Hello World\n"; // Write: text + newline
12     }
13

```

```

14     std::ifstream in(path);
15     int num = 0;
16     std::string line1;
17     std::string line2;
18
19     // [KEY] >> reads 42 then stops, but leaves newline \n in input buffer
20     in >> num;
21
22     // [TRAP] getline reads the remaining \n (empty line), not "Hello World"
23     std::getline(in, line1);
24
25     // This one actually reads "Hello World"
26     std::getline(in, line2);
27
28     std::cout << "num=" << num << '\n';
29     std::cout << "first getline len=" << line1.size() << " repr=\"" << line1
30     << "\"\n";
31     std::cout << "second getline=\"" << line2 << "\"\n";
32     return 0;
33 }

```

Syntax / trap walkthrough:

Step	Consumed	Left in buffer	Issue
<code>in >> num</code>	42	<code>\nHello World\n</code>	Stops before whitespace; newline stays in the buffer.
First <code>getline</code>	<code>\n</code> (empty string)	<code>Hello World\n</code>	Immediately hits newline; you get an empty line.
Second <code>getline</code>	Hello World	(empty)	Correct text, but you are “one line off” from what you intended.

Root cause: `>>` is *formatted extraction*: it stops at whitespace but does not consume it. `getline` reads from the current position up to the next newline.

Fix: see the next example (``ex04b``).

Example 4b — Fix with ``ignore()`` (``ex04b_extract_then_getline_fix.cpp``)

Build: ``g++ -std=c++17 -Wall -o ex04b ex04b_extract_then_getline_fix.cpp``

Code block

```
1  // ex04b_extract_then_getline_fix.cpp
2  #include <fstream>
3  #include <iostream>
4  #include <limits>    // [KEY] Provides numeric_limits required for max()
5  #include <string>
6
7  int main() {
8      const char* path = "mixed_ex04b.txt";
9      {
10         std::ofstream out(path);
11         out << "42\n";
12         out << "Hello World\n";
13     }
14
15     std::ifstream in(path);
16     int num = 0;
17     std::string line;
18
19     in >> num;
20
21     // [KEY FIX] ignore() skips until delimiter (2nd param), or max count (1st
    param)
22     // numeric_limits<streamsize>::max() = as many as possible, until newline
    found
23     in.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
24
25     // Now getline starts from "Hello World", reads correctly
26     std::getline(in, line);
27
28     std::cout << "num=" << num << '\n';
```

```

29     std::cout << "getline=\"" << line << "\"\n";
30     return 0;
31 }

```

Key syntax:

API	Form	Role	Note
<code>ignore(n, delim)</code>	Skip up to <code>n</code> chars until <code>delim</code>	Clears leftover input after <code>>></code>	The delimiter is extracted and discarded.
<code>numeric_limits<T>::max()</code>	Largest value of <code>T</code>	Use as “read until delimiter” count	Requires <code>#include <limits></code> .

Fix in steps:

1. After `in >> num`, the buffer is `\nHello World\n`.
2. `ignore(max, '\n')` discards through and including the first `\n`.
3. Buffer is now `Hello World\n`.
4. `getline` reads `Hello World` as intended.

Alternative: if you only need numbers, avoid mixing `>>` and `getline`, or read everything with `getline` and parse with `stoi` / `stod`.

Example 5 — Name with spaces (`ex05\_name\_with\_spaces.cpp`)

Chess link: player handle `"Anna Lee"` + numeric rating.

Build: `g++ -std=c++17 -Wall -o ex05 ex05_name_with_spaces.cpp`

```

1 // ex05_name_with_spaces.cpp
2 #include <fstream>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     const char* path = "player_ex05.txt";
8     {
9         std::ofstream out(path);
10        out << "Anna Lee\n";    // String with spaces
11        out << "1600\n";        // Pure number
12    }
13
14    std::ifstream in(path);
15    std::string name;
16    int rating = 0;
17
18    // [KEY] getline reads entire line including spaces, until newline
19    if (!std::getline(in, name)) {
20        std::cerr << "read name failed\n";
21        return 1;
22    }
23    // Buffer state: \n consumed, next is "1600\n"
24
25    // [KEY] Use >> to read integer from next line
26    if (!(in >> rating)) {
27        std::cerr << "read rating failed\n";
28        return 1;
29    }
30
31    std::cout << "Name: \"" << name << "\" Elo: " << rating << '\n';
32    return 0;
33 }

```

Syntax comparison:

Method	Reads	Stops at	Good for
<code>in >> str</code>	One whitespace-delimited token	Space, tab, newline	Single words without spaces
<code>getline(in, str)</code>	One full line	<code>\n</code> (consumed, not stored in <code>str</code>)	Names or sentences with spaces

File layout strategy:

- Store one text line, then one numeric line, in a fixed repeating pattern.
- Read order must **exactly** match write order.
- Use `getline` for a full line of text and `>>` for a lone number on its own line; do not mix both on the same line.

Error-handling pattern:

- Check each read; on failure return an error code (or `false`) immediately.
- Use `if (!(stream >> var))` to test stream state.

Example 6 — Versioned save with `vector` (`ex06_versioned_savegame.cpp`)

Code block

```
1 // ex06_versioned_savegame.cpp
2 #include <fstream>
3 #include <iostream>
4 #include <vector>
5
6 struct SaveGameV1 {
7     int level = 1; // C++11: member default initialization
8     std::vector<int> inventory_ids;
9 };
10
11 void save(const SaveGameV1& g, const char* path) {
12     std::ofstream out(path);
13     // [KEY] Serialization protocol: one field per line, write version first
14     // (future compatibility)
15     out << 1 << '\n'; // Line 1: format version
16     out << g.level << '\n'; // Line 2: scalar value
17     out << g.inventory_ids.size() << '\n'; // Line 3: vector length
18     // [KEY] Write elements one by one, not whole vector (vector has internal
19     // pointers, can't write raw)
20     for (int id : g.inventory_ids)
21         out << id << '\n'; // Line 4+: each element on its own line
```

```

20 }
21
22 bool load(SaveGameV1& g, const char* path) {
23     std::ifstream in(path);
24     if (!in)
25         return false;           // File not existing is not a fatal error
26
27     int ver = 0;
28     in >> ver;
29     if (ver != 1)
30         return false;           // [KEY] Version mismatch, reject load
                                   (prevent data corruption)
31
32     in >> g.level;
33     std::size_t n = 0;
34     in >> n;                     // Read element count
35
36     g.inventory_ids.resize(n);   // [KEY] Pre-allocate space, avoid multiple
                                   reallocations
37     for (std::size_t i = 0; i < n; ++i)
38         in >> g.inventory_ids[i]; // Fill directly into reserved slots
39
40     // [KEY] Cast to bool to test stream state: false means read error
41     return static_cast<bool>(in);
42 }
43
44 int main() {
45     const char* path = "save_ex06.txt";
46
47     SaveGameV1 a;
48     a.level = 7;
49     a.inventory_ids = {10, 20, 30};
50     save(a, path);
51
52     SaveGameV1 b;
53     if (!load(b, path)) {       // [KEY] Always check return value
54         std::cerr << "load failed\n";
55         return 1;
56     }
57     std::cout << "level=" << b.level << " items:";
58     for (int x : b.inventory_ids)
59         std::cout << ' ' << x;
60     std::cout << '\n';
61     return 0;
62 }

```

Syntax / design patterns:

Technique	Code	Role
Version field	<code>out << 1 << '\n'</code>	Room to evolve the save format later.
Length prefix	<code>out << vec.size()</code>	Loader knows how many elements follow.
<code>resize</code> then fill	<code>vec.resize(n)</code>	One allocation instead of many <code>push_back</code> reallocations.
Stream as bool	<code>static_cast<bool>(in)</code>	Concise “no failure” test (similar to <code>!in.fail()</code>).
Default member init	<code>int level = 1;</code>	C++11: members get values before the constructor body runs.

Serialization rules:

- Never `write(&vec, sizeof(vec))` — a `vector` holds pointers; you would save meaningless addresses.
 - Scalars can be text (`<<`) or binary (`write`), but aggregates must be serialized field by field.
- Protocol: whatever order you write, read in the **same** order.

Example 7 — Binary POD struct (`ex07_binary_pod_struct.cpp`)

Rule: only trivial layout — good for a simple `struct` with `int` / `float`, not `std::string`.

Build: `g++ -std=c++17 -Wall -o ex07 ex07_binary_pod_struct.cpp`

```

1 // block07_binary_pod_struct.cpp
2 #include <fstream>
3 #include <iostream>
4
5 // [KEY] POD = Plain Old Data, only basic types, no virtual
  functions/pointers/string
6 struct PlayerPod {
7     int id = 0;
8     float x = 0.f;
9     float y = 0.f;
10 };
11
12 int main() {
13     const char* path = "player_ex07.bin";
14
15     {
16         PlayerPod p{1, 100.f, 200.f}; // C++11: Uniform brace initialization
17         std::ofstream out(path, std::ios::binary);
18
19         // [KEY] write(address, byte_count), writes entire struct as-is to file
20         // &p → reinterpret_cast<const char*> → byte sequence
21         out.write(reinterpret_cast<const char*>(&p), sizeof(p));
22     }
23
24     PlayerPod q{}; // Value-initialization, all members zeroed
25     {
26         std::ifstream in(path, std::ios::binary);
27         // [KEY] read(target_address, byte_count), reads from file back to
memory
28         // reinterpret_cast<char*>: note no const needed here since we're
writing to memory
29         in.read(reinterpret_cast<char*>(&q), sizeof(q));
30     }
31
32     std::cout << "Loaded id=" << q.id << " pos=(" << q.x << ", " << q.y <<
"\n";
33     return 0;
34 }

```

Key syntax:

API	Arguments	Meaning
-----	-----------	---------

<code>write(ptr, count)</code>	Source address, byte count	Copy a memory block verbatim into the file.
<code>read(ptr, count)</code>	Destination address, byte count	Copy bytes from the file into memory.
<code>sizeof(p)</code>	Compile-time	Total bytes of <code>PlayerPod</code> (often 12 on typical platforms: one <code>int</code> plus two <code>float</code> s; sizes are implementation-defined).
<code>reinterpret_cast</code>	Type change	Changes how the pointer is interpreted, not the underlying bits.

POD / trivial-layout checklist:

- OK: only scalars like `int` / `float` / `double`, or structs of such types.
- OK: no user-defined constructors, virtual functions, or inheritance.
- Not OK: `std::string` (heap pointer inside).
- Not OK: `std::vector` (internal dynamic pointers).
- Not OK: polymorphic classes (vtable pointer layout).

Risks:

- Binary layouts are not portable (endianness, alignment, type sizes).
 - Adding fields to the struct breaks old saves unless you version the format.
- Prefer **versioned text saves** (like ``ex06``) for game data when you can.

Example 8 — Whole file into ``std::string`` (``ex08_whole_file_std.cpp``)

Teaches the “load entire file” idea in pure C++. For **raylib** ``LoadFileText`` / ``UnloadFileText``, see ``file_IO.md`` §5 inside your raylib project.

Run from `ch9\_lab\_examples/` so `hello.txt` is found.

Build: `g++ -std=c++17 -Wall -o ex08 ex08\_whole\_file\_std.cpp`

Code block

```
1  // ex08_whole_file_std.cpp
2  #include <fstream>
3  #include <iostream>
4  #include <sstream> // [KEY] ostringstream for in-memory string building
5  #include <string>
6
7  std::string load_file_as_string(const char* path) {
8      std::ifstream in(path, std::ios::binary);
9      if (!in)
10         return {}; // [KEY] Return empty string to indicate
// failure (nullopt style)
11
12     std::ostringstream oss; // String stream: writes like cout, but
// target is in memory
13
14     // [KEY SYNTAX] in.rdbuf() returns the stream's internal buffer pointer
// (streambuf*)
15     // oss << streambuf* is an overload that copies entire file content to oss
16     oss << in.rdbuf();
17
18     return oss.str(); // Extract all accumulated content from oss
// as std::string
19 }
20
21 int main() {
22     const char* path = "hello.txt";
23     std::string s = load_file_as_string(path);
24     if (s.empty()) { // [NOTE] Both empty file and failure return
// empty, real projects need differentiation
25         std::cerr << "Could not read " << path << '\n';
26         return 1;
27     }
28     std::cout << "---- file begin ----\n" << s << "---- file end ----\n";
29     return 0;
30 }
```

Key syntax:

Type / API	Role	Analogy
<code>std::ostringstream</code>	In-memory output stream	Like <code>std::cout</code> , but builds a <code>std::string</code> .
<code>in.rdbuf()</code>	Access the stream’ s buffer	The “pipe” into the file’ s bytes.
<code>oss << in.rdbuf()</code>	Bulk copy	Faster than a character-by-character loop.
<code>oss.str()</code>	Get the finished string	Drain everything written into the stream.

****vs raylib `LoadFileText`:****

Approach	Pros	Cons
This example (standard library)	Pure C++, no extra deps, RAI <code>std::string</code>	Large files allocate a second full copy in memory.
raylib <code>LoadFileText</code>	Simple, tuned API	Must pair with <code>UnloadFileText</code> ; manual C-string lifetime.

Typical uses: config files, small text assets, unit tests that assert on file contents.

Example 9 — One-line FEN save/load (`ex9\_fen\_line\_io.cpp`)

Chess link: persist a **standard one-line FEN** (the usual starting position in this lab) as plain text.

What is FEN? (Forsyth–Edwards Notation)

FEN is a *text standard* for describing a *complete chess position* in a single line. Engines, databases, and websites use it to exchange board states. For this course, it is a realistic “long string” to practice **text file save/load** with `std::getline` (spaces are part of the data, so line-based I/O fits well).

A legal FEN string has **six fields**, separated by **spaces**:

#	Field	Typical example (start position)	Meaning (short)
---	-------	----------------------------------	-----------------

| 1 | **Piece placement** | ``rnbqkbnr/pppppppp/8/8/PPPPPPPP/RNBQKBNR`` | Eight *ranks* from Black’s back rank (8) down to White’s back rank (1). Within each rank, read left→right (files **a–h**). A `**`/`**` starts the next lower rank. **Uppercase** letters are White pieces, **lowercase** are Black. ****Digits `1`–`8`**** mean “that many empty squares” in a row. |

| 2 | **Active color** | ``w`` | Side to move: `**`w`**` = White, `**`b`**` = Black. |

| 3 | **Castling availability** | ``KQkq`` | Which king/rook pairs may still castle from their *original* corners (``K`/`Q`` = White kingside/queenside; ``k`/`q`` = Black). Use `**`-`**` if nobody can castle anymore. |

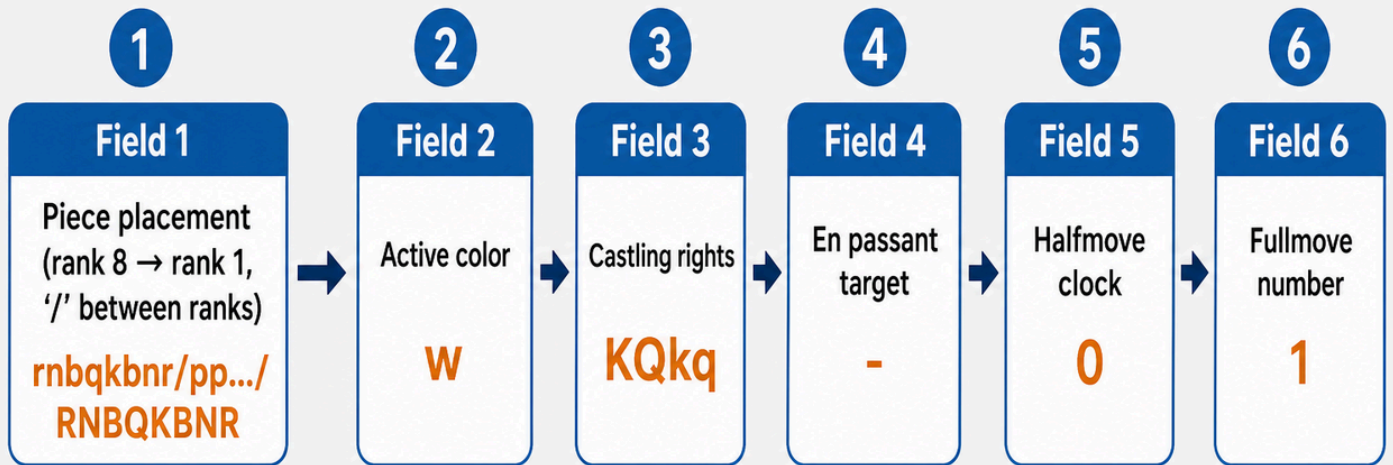
| 4 | **En passant target square** | ``-`` | If the last move was a pawn double-step, this is the square *behind* that pawn (where an en passant capture could land); otherwise `**`-`**`. |

| 5 | **Halfmove clock** | ``0`` | Count of half-moves since the last pawn move or capture (used for the 50-move rule in serious play). |

| 6 | **Fullmove number** | ``1`` | Starts at **1**; in standard rules it increments after Black’s move. |

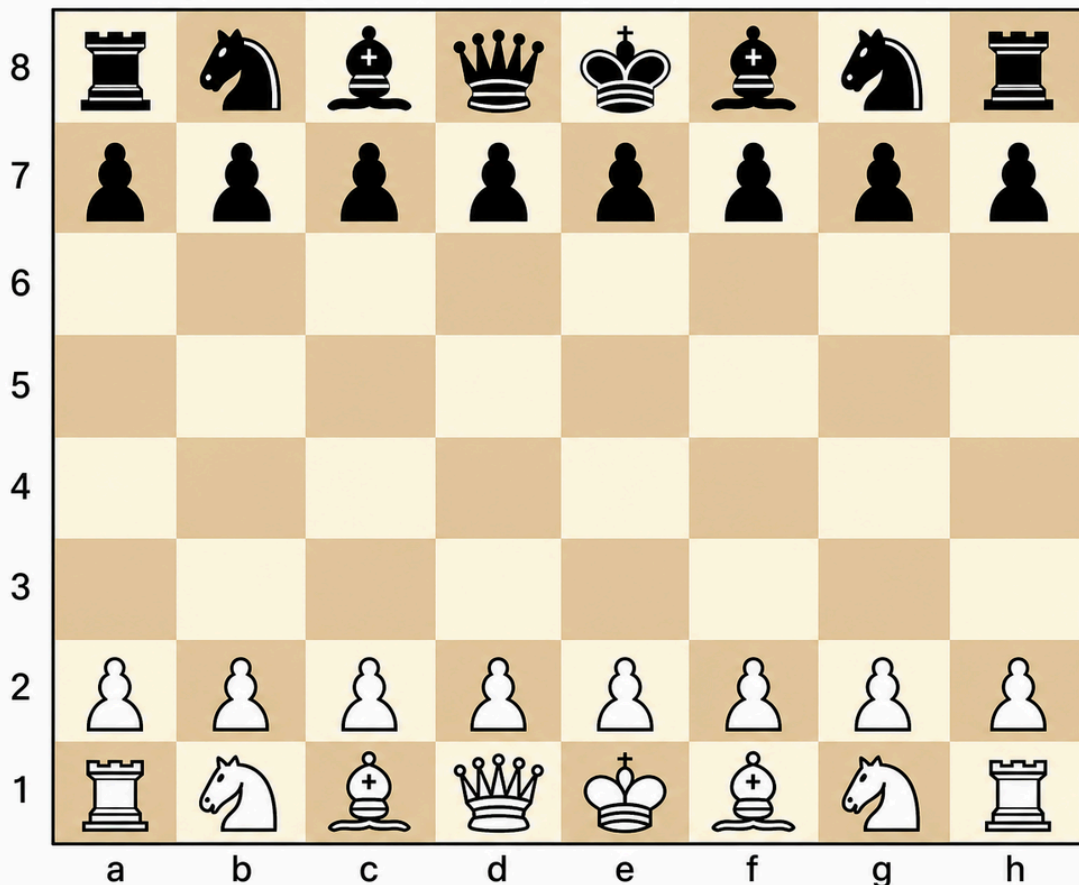
Piece letters in field 1: ``P/p`` pawn, ``R/r`` rook, ``N/n`` knight, ``B/b`` bishop, ``Q/q`` queen, ``K/k`` king.

FEN — six fields (space-separated)



i Uppercase = White pieces, lowercase = Black; digits = empty squares

FEN piece placement: read from rank 8 to rank 1



```

1 // board_ex10_fen_line_io.cpp
2 #include <fstream>
3 #include <iostream>
4 #include <string>
5
6 bool save_fen(const char* path, const std::string& fen) {
7     std::ofstream out(path);
8     if (!out)
9         return false;
10    out << fen << '\n';
11    return true;
12 }
13
14 bool load_fen(const char* path, std::string& fen) {
15     std::ifstream in(path);
16     if (!std::getline(in, fen))
17         return false;
18     return true;
19 }
20
21 int main() {
22     const char* path = "board_ex10.fen";
23     const std::string start =
24         "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1";
25
26     if (!save_fen(path, start)) {
27         std::cerr << "save failed\n";
28         return 1;
29     }
30     std::string loaded;
31     if (!load_fen(path, loaded)) {
32         std::cerr << "load failed\n";
33         return 1;
34     }
35     std::cout << "FEN loaded OK, length = " << loaded.size() << '\n';
36     return 0;
37 }

```

Key Syntax & Design:

Function	Return Type	Parameter Passing	Purpose

<code>save_fen</code>	<code>bool</code>	<code>const std::string&</code> (const reference)	Returns success/failure; uses reference to avoid copying long string
<code>load_fen</code>	<code>bool</code>	<code>std::string&</code> (non-const reference)	Passes output parameter by reference; writes result to caller's variable
<code>std::getline</code>	<code>bool</code>	stream + string reference	Reads entire line including spaces; returns false on EOF or error

Reference Passing Patterns:

- `const std::string& fen` (input): read-only, no copy, efficient for large strings
- `std::string& fen` (output): allows function to write result back to caller

Why use output parameter instead of return value?

Code block

```

1  // Option A: return by value (copies the string)
2  std::string load_fen(const char* path) { ... }
3
4  // Option B: output parameter (no copy, more efficient)
5  bool load_fen(const char* path, std::string& result) { ... }
```

Error Handling Strategy:

- Always check file open before writing
- `getline` returns bool: test it immediately
- Return `false` on any failure so caller can decide what to do

FEN recap: The program stores exactly **one line** containing all six fields (see *What is FEN?* and Figures 1–2 above). That matches `out << fen << '\n'` plus `std::getline` on load: portable, human-readable, and easy to inspect in a text editor.

Part B — MVC Architecture and Game Design (30 min)

B1. The Problem: Spaghetti Code

Typical beginner raylib game:

Code block

```
1  int main() {
2      InitWindow(800, 600, "Game");
3      int playerX = 400, playerY = 300, score = 0;
4
5      while (!WindowShouldClose()) {
6          // Input + Logic + Drawing ALL MIXED
7          if (IsKeyDown(KEY_RIGHT)) playerX += 5;
8          if (IsKeyDown(KEY_LEFT))  playerX -= 5;
9
10         if (CheckCollision(playerX, playerY, coinX, coinY)) {
11             score += 10; // Logic buried in game loop
12             coinX = GetRandomValue(0, 700);
13         }
14
15         BeginDrawing();
16         ClearBackground(RAYWHITE);
17         DrawRectangle(playerX, playerY, 40, 40, BLUE); // Drawing in main
18         DrawText(TextFormat("Score: %i", score), 10, 10, 20, BLACK);
19         EndDrawing();
20     }
21 }
```

Problems:

- Cannot unit test `score += 10` without initializing a window
- Change drawing order → may break collision logic
- Add a second player → copy-paste variables everywhere
- Save game state → hard to extract what's "data" vs "temporary"

B2. MVC: Separation of Concerns

MVC splits a program into three independent parts:

Component	Responsibility	What it KNOWS	What it DOES	Chess Analogy
-----------	----------------	---------------	--------------	---------------

| **Model** | Data + Rules | Game state (positions, scores, whose turn) | Enforce rules, update state |
The chess position + legal move rules |

| **View** | Presentation | Nothing about rules; only how to draw | Render model on screen | The
physical board and pieces you see |

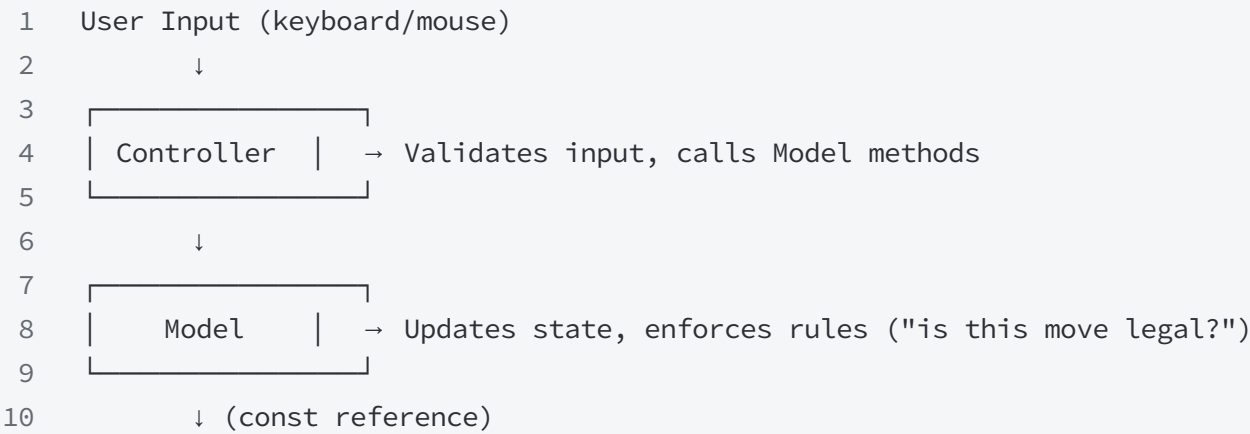
| **Controller** | Input Handling | User actions (clicks, keys) | Translate input → model commands |
The player moving pieces |

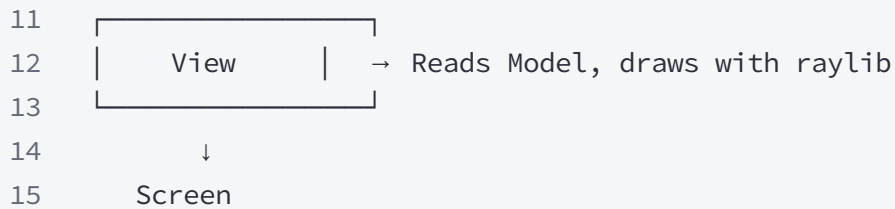
Key principle:

- **Model** knows **nothing** about graphics (no ``Color``, no ``DrawRectangle``)
- **View** reads **only** from Model (const reference), never modifies it
- **Controller** is the **only** part that modifies Model, through well-defined methods

B3. Data Flow Diagram

Code block





Golden Rules:

1. **View never calls Model setters** — only getters
2. **Model never calls raylib** — pure C++ data and logic
3. **Controller never draws** — only handles input and calls Model

B4. MVC in raylib: "Eat the Coin" Example

Game: Player (blue square) moves with arrows to collect gold coins. Each coin = +10 points.

4.1 Model (`GameModel`)

Code block

```
1  // GameModel.h - NO raylib includes!
2  #pragma once
3
4  class GameModel {
5  public:
6      // Constants (game rules)
7      static constexpr int PLAYER_SIZE = 40;
8      static constexpr int COIN_RADIUS = 15;
9      static constexpr int WINDOW_W = 800;
10     static constexpr int WINDOW_H = 600;
11     static constexpr int SPEED = 5;
12
13     // Constructor sets initial state
14     GameModel();
15
```

```

16 // Queries (View uses these)
17 int getPlayerX() const { return playerX_; }
18 int getPlayerY() const { return playerY_; }
19 int getCoinX() const { return coinX_; }
20 int getCoinY() const { return coinY_; }
21 int getScore() const { return score_; }
22
23 // Commands (Controller calls these)
24 void movePlayer(int dx, int dy); // Clamps to window bounds
25 void update(); // Checks collision, respawns coin
26 void saveToFile(const char* path) const; // Uses fstream
27 bool loadFromFile(const char* path); // Returns success
28
29 private:
30     int playerX_, playerY_;
31     int coinX_, coinY_;
32     int score_;
33
34     bool checkCollision() const;
35     void respawnCoin();
36 };

```

Code block

```

1 // GameModel.cpp
2 #include "GameModel.h"
3 #include <algorithm> // std::clamp
4 #include <cstdlib> // rand
5 #include <fstream>
6
7 GameModel::GameModel()
8     : playerX_(WINDOW_W/2), playerY_(WINDOW_H/2),
9       coinX_(200), coinY_(200), score_(0) {
10     respawnCoin();
11 }
12
13 void GameModel::movePlayer(int dx, int dy) {
14     playerX_ += dx;
15     playerY_ += dy;
16     // Enforce boundary rules
17     playerX_ = std::clamp(playerX_, 0, WINDOW_W - PLAYER_SIZE);
18     playerY_ = std::clamp(playerY_, 0, WINDOW_H - PLAYER_SIZE);
19 }
20

```

```

21  bool GameModel::checkCollision() const {
22      int px = playerX_ + PLAYER_SIZE/2;
23      int py = playerY_ + PLAYER_SIZE/2;
24      int dx = px - coinX_;
25      int dy = py - coinY_;
26      int distSq = dx*dx + dy*dy;
27      int threshold = (PLAYER_SIZE/2 + COIN_RADIUS);
28      return distSq <= threshold * threshold;
29  }
30
31  void GameModel::update() {
32      if (checkCollision()) {
33          score_ += 10;           // Rule: coin gives 10 points
34          respawnCoin();         // Rule: coin moves randomly
35      }
36  }
37
38  void GameModel::respawnCoin() {
39      coinX_ = rand() % (WINDOW_W - 2*COIN_RADIUS) + COIN_RADIUS;
40      coinY_ = rand() % (WINDOW_H - 2*COIN_RADIUS) + COIN_RADIUS;
41  }
42
43  // Save: version 1 format
44  void GameModel::saveToFile(const char* path) const {
45      std::ofstream out(path);
46      out << 1 << '\n';           // version
47      out << playerX_ << ' ' << playerY_ << '\n';
48      out << coinX_ << ' ' << coinY_ << '\n';
49      out << score_ << '\n';
50  }
51
52  bool GameModel::loadFromFile(const char* path) {
53      std::ifstream in(path);
54      if (!in) return false;
55      int ver;
56      in >> ver;
57      if (ver != 1) return false;
58      in >> playerX_ >> playerY_;
59      in >> coinX_ >> coinY_;
60      in >> score_;
61      return static_cast<bool>(in);
62  }

```

4.2 View (`GameView`)

Code block

```
1 // GameView.h
2 #pragma once
3 #include "GameModel.h"
4 #include "raylib.h"
5
6 class GameView {
7 public:
8     // Takes const reference - View cannot modify Model!
9     void draw(const GameModel& model) const;
10
11 private:
12     void drawPlayer(const GameModel& m) const;
13     void drawCoin(const GameModel& m) const;
14     void drawUI(const GameModel& m) const;
15 };
```

Code block

```
1 // GameView.cpp
2 #include "GameView.h"
3
4 void GameView::draw(const GameModel& model) const {
5     ClearBackground(RAYWHITE);
6     drawCoin(model);
7     drawPlayer(model);
8     drawUI(model);
9 }
10
11 void GameView::drawPlayer(const GameModel& m) const {
12     DrawRectangle(
13         m.getPlayerX(),
14         m.getPlayerY(),
15         GameModel::PLAYER_SIZE,
16         GameModel::PLAYER_SIZE,
17         BLUE
18     );
19 }
20
21 void GameView::drawCoin(const GameModel& m) const {
22     DrawCircle(
23         m.getCoinX(),
```

```

24         m.getCoinY(),
25         GameModel::COIN_RADIUS,
26         GOLD
27     );
28 }
29
30 void GameView::drawUI(const GameModel& m) const {
31     DrawText(
32         TextFormat("Score: %i", m.getScore()),
33         10, 10, 20, BLACK
34     );
35     DrawText("Press S to save, L to load", 10, 40, 20, DARKGRAY);
36 }

```

4.3 Controller (` GameController `)

Code block

```

1  // GameController.h
2  #pragma once
3  #include "GameModel.h"
4
5  class GameController {
6  public:
7      // Process one frame of input
8      void handleInput(GameModel& model);
9
10     // Called every frame (handles timing, AI, etc.)
11     void update(GameModel& model);
12 };

```

Code block

```

1  // GameController.cpp
2  #include "GameController.h"
3  #include "raylib.h"
4
5  void GameController::handleInput(GameModel& model) {
6      int dx = 0, dy = 0;
7      if (IsKeyDown(KEY_RIGHT)) dx += GameModel::SPEED;

```

```

8     if (IsKeyDown(KEY_LEFT)) dx -= GameModel::SPEED;
9     if (IsKeyDown(KEY_DOWN)) dy += GameModel::SPEED;
10    if (IsKeyDown(KEY_UP))    dy -= GameModel::SPEED;
11
12    // Only Controller modifies Model!
13    if (dx != 0 || dy != 0)
14        model.movePlayer(dx, dy);
15
16    // Save/Load triggered by Controller
17    if (IsKeyPressed(KEY_S))
18        model.saveToFile("savegame.txt");
19    if (IsKeyPressed(KEY_L))
20        model.loadFromFile("savegame.txt");
21 }
22
23 void GameController::update(GameModel& model) {
24     // Run game logic (collision, scoring)
25     model.update();
26 }

```

4.4 Main (wires everything together)

Code block

```

1  // main.cpp
2  #include "raylib.h"
3  #include "GameModel.h"
4  #include "GameView.h"
5  #include "GameController.h"
6
7  int main() {
8      InitWindow(GameModel::WINDOW_W, GameModel::WINDOW_H, "MVC Demo");
9      SetTargetFPS(60);
10
11     GameModel model;           // Pure data + rules
12     GameView view;             // Pure drawing
13     GameController ctrl;       // Pure input handling
14
15     while (!WindowShouldClose()) {
16         // Phase 1: Input → Model updates
17         ctrl.handleInput(model);
18         ctrl.update(model);
19     }

```

```

20      // Phase 2: View renders (never modifies)
21      BeginDrawing();
22      view.draw(model);
23      EndDrawing();
24  }
25
26      CloseWindow();
27      return 0;
28  }

```

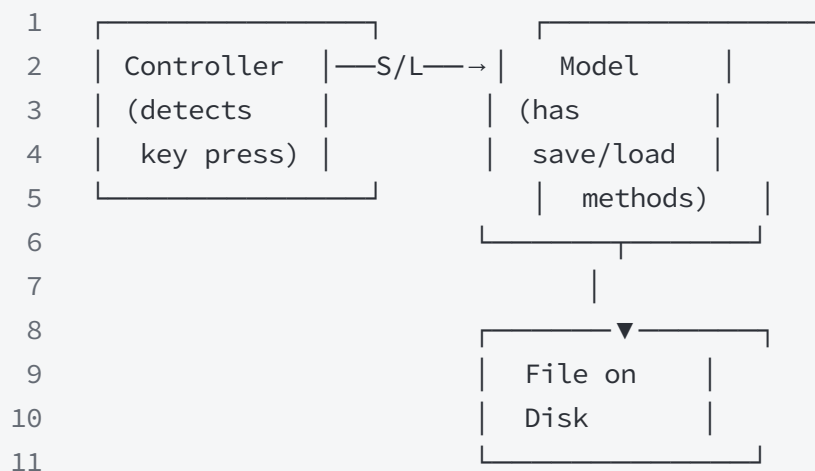
B5. MVC Benefits (Why bother?)

Without MVC	With MVC
Test collision by launching window	Test <code>model.update()</code> in console unit test
Change blue to green → search whole file	Change one line in <code>drawPlayer()</code>
Add save/load → spaghetti code	Add <code>saveToFile()</code> in Model, trigger from Controller
Port to console → rewrite everything	Write <code>ConsoleView</code> that prints numbers
Two players → copy-paste mess	Create <code>Player</code> class in Model, View draws vector

B6. MVC + File I/O Integration

Where does save/load fit?

Code block



Rule: View never touches files. Controller initiates, Model knows the format.

In our raylib example:

- Press **S** → Controller calls `model.saveToFile("savegame.txt")``
- Press **L** → Controller calls `model.loadFromFile("savegame.txt")``
- Model writes: version, player pos, coin pos, score (one per line)

B7. Common MVC Mistakes

Mistake	Why Wrong	Correct
<code>Texture2D</code> inside Model	Model depends on raylib, hard to test	Store <code>int textureID</code> , load in View
View calls <code>model.setScore(999)</code>	View modifies state → bugs	Controller validates, then calls setter
Controller calls <code>DrawText</code>	Mixes input and rendering	Controller only modifies Model; View draws

One giant class "Game" with everything	No separation	Three distinct classes with clear interfaces
--	---------------	--

B8. Summary: MVC in 3 Lines

- > **Model** = What is true? (positions, scores, rules)
- > **View** = What do you see? (colors, shapes, text)
- > **Controller** = What did the user do? (keys, clicks → commands)

Next step: Take your raylib project, separate variables into ``Model``, move ``Draw*`` calls into ``View``, put input handling in ``Controller``. The save/load from Part A plugs right into the Model.